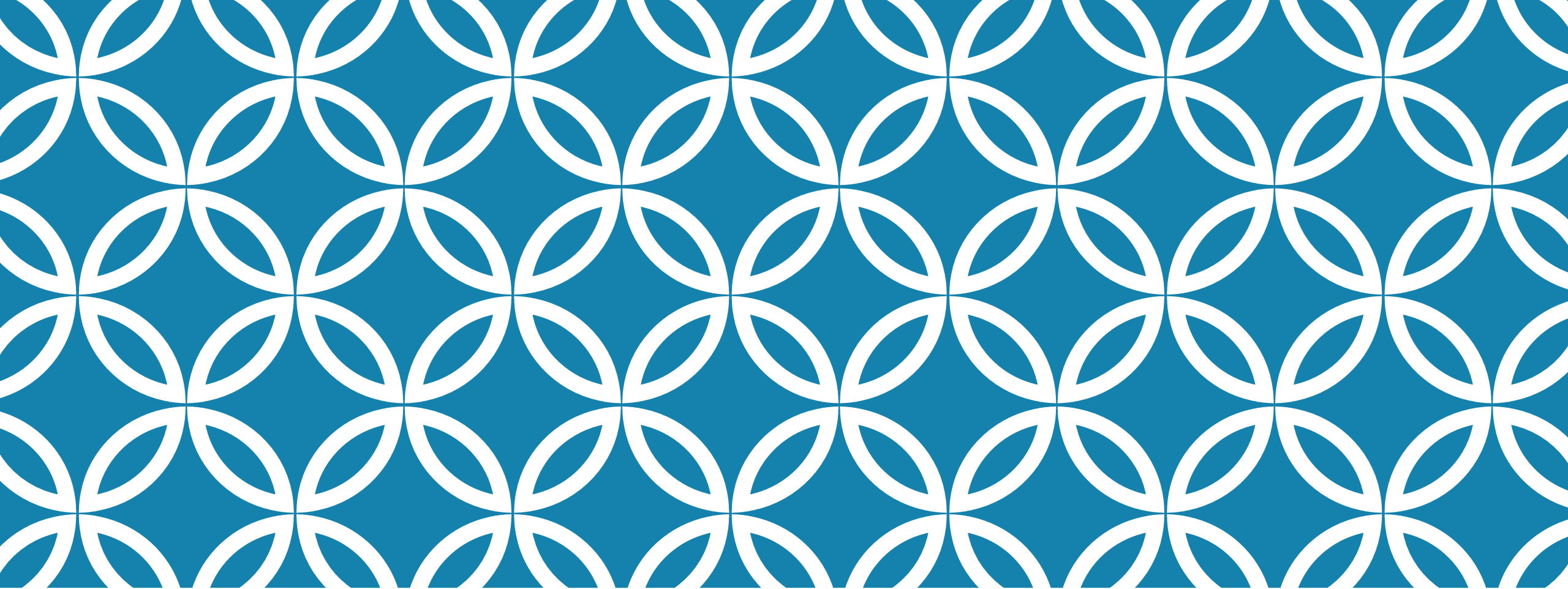




CYBERLAND SOLUTION



STATEMENT

Given a undirected weighted graph with N nodes and M edges. Some nodes can clear the current distance to 0, while some can divide the distance by 2. In each visit, you can only use the special ability at most once, and you can choose not to use the ability on the node.

Find the shortest path from 0 to H , if you can not visit H twice, and the divide-by-2 ability can be used at most K times.

$N, M \leq 10^5, K \leq 10^6$

STATEMENT?

Given a undirected weighted graph with N nodes and M edges. Some nodes can clear the current distance to 0, while some can divide the distance by 2. In each visit, you can only use the special ability at most once, and **you must use the ability** on the node.

Find the shortest path from 0 to H , if you can not visit H twice, and the divide-by-2 ability can be used at most K times.

$N, M \leq 10^5, K \leq 30$

SUBTASK 1

$n \leq 3$

Either go directly H, or go to H after passing through the third node.

Because O, H has no special abilities, it is obviously not beneficial to repeatedly visit the third point.

Time complexity $O(1)$

SUBTASK 2,5

No Special ability。

A simple single source shortest path(SSSP) problem which can be solved through Dijkstra algorithm.

Time Complexity $O(M \log n)$ 。

SUBTASK 3,6

No divide-by-2 ability.

Find the set of node S , which have set-zero ability, and can be reached from 0 without go through H.

If we reach any node in S , we can clear the distance to 0. So it's a multiple source shortest path problem with $\{S\} \cup \{0\}$ as the source set.

Use Dijkstra again and the time complexity is also $O(M \log N)$.

SUBTASK 4,7

$K \leq 30$

Same as the idea of subtask 3,6, we can find the set S , and treat the problem as a multi-source shortest path problem.

Since $K \leq 30$, we can use dynamic programming, and the state go as follows:

$f[i][t]$: The shortest distance to node i , if exactly t time of special ability is used.

To transfer between states, we need to consider whether to use the special ability.

- If we do not use special ability, we can treat transition as another multi-source shortest path problem.
- If we use special ability, t is increased by 1, we can transfer in order of increasing t .

Time complexity $O(MK \log N)$.

SUBTASK 8

$K \leq 1000000$ 。

Suspect that K is not required to be very large.

Accepted.

WHY?

We prove that for every possible V , assume that we can use at most V times of divide-by-2 ability, we can always make the absolute errors at most $N \max(C) / 2^K$, If we use the algorithm for subtask 7.

Here, $\max(C)$ represents the maximum cost to pass through an edge.

WHY?

1. Assume that the optimal solution for at most i -times divide-by-2 ability usage is $\text{ans}[i]$. Then $\text{ans}[i]$ is non-increasing.
2. Find the optimal solution of at most V -times divide-by-2 ability usage ($V \gg K$ holds). We call this the original solution.
3. Keep the last K divide-by-two ability usage of the original solution, and delete all previous divide-by-two ability usage. We call this the intermediate solution.
4. Assuming that the intermediate path uses divided-by-two ability first in node p . Replace the path before first divided-by-two ability usage, with the shortest path from 0 to p without using divide-by-2 ability. We call this path the edited solution.
5. The shortest path from 0 to p is at most $N \max\{C\}$, and edited solution preserves the path of the original solution after the last K divide-by-two ability usage. So it is at most $N \max\{C\} / 2^K$ worse than the original solution.

NOTE

Take $K = \min\{K, 70\}$, and an absolute error of at most $1e-7$ can be achieved.

Time complexity $O(m \log n (\log n + \log \max\{V\} - \log \epsilon))$.

Sequence

Subtask1

- We can use a simple brute force algorithm.
- For example, enumerate over every possible (l, r) and sort(or nth_element) the sequence to find the median. Then count the occurrence.
- Time complexity: $O(n^3) - O(n^3 \log n)$.

Subtask2

- Here we need to find medians of a sequence quickly.
- A simple way is to use two heaps to maintain the bigger part and the smaller part of the sequence. Or just use a balanced tree.
- Time complexity $O(n^2 \log n)$.
- There's also another way, reverse the operations, then we just need to delete an existing value and find the medians. This can be done with a linked list in $O(n^2)$.

Subtask3

- For every value x , occurrences of x form two continuous subsequence.
- If we choose x as the final median. We either choose one of the subsequence or choose both if possible.
- It's easy to determine whether it's possible to choose both.

Subtask4

- If the median of the whole sequence is not 2, then it appears in the sequence more than $n/2$, and that's of course the answer. For the rest of the cases, choosing $l = 0, r = n - 1, x = 2$ is the best choice if $x = 2$.
- Then just consider $x = 1, 3$. If we choose $x = 1$, we can see every 1 as 1, and every 2, 3 as -1. Then we want to find a continuous subsequence with non-negative sum and with the most 1.
- Do a prefix sum, then for every r find the smallest l satisfying $sum_r \geq sum_l$.

Subtask5

- We just need to determine whether the answer is 1 or 2.
- For every value x that appeared twice at x_1, x_2 :
- Construct a sequence $B_i = \max(-1, \min(A_i - x, 1))$.
- If we can find $l \leq x_1 \leq x_2 \leq r$ with sum $[-2, 2]$, then x is a median of such (l, r) .
- Find $l \leq x_1 \leq x_2 \leq r$ with the biggest sum and smallest sum, call these sums v_1, v_2 . Since moving l or r by one step only affect the sum by at most 1, so every integer sum in $[v_2, v_1]$ is achievable.

Subtask 6&7

- From the solution of Subtask5, we can actually determine if there is (l, r) satisfying $l \leq x_1 \leq x_2 \leq r$ and has median x .
- View every $z \geq x$ as 1, others as -1, find (l, r) with maximum sum, call this v_1 .
- View every $z > x$ as 1, others as -1, find (l, r) with minimum sum, call this v_2 .
- If $v_1 * v_2 \leq 0$, then there exist such (l, r) .

Subtask 6&7

- Enumerate x from small to big.
- For the occurrences of x . Run two pointers to see if there is a sequence with median x and contains every x from l to r . The checking is described before.
- Time complexity: $O(n \log n)$.

Alice, Bob and Circuit

Task Review

- Original task
 - n persons, each with a name and a number.
 - m letters, each specified by a sender and a recipient's names, with the content of the sender's number.
 - For each person, calculate the sum of the received letter contents (modulo 2^{16})
 - $n \leq 700, m \leq 1000$
- Communication style
 - Alice knows the information of n persons, and Bob knows that of m letters. Each needs to send a binary string within 10^5 bits to Circuit.
 - Circuit has to use at most 2×10^7 binary logic gates to get the results.

Subtask 1

- $n = 1, m = 0$
- No letters are sent. The result is simply 0.
 - 0 can be generated by a gate of operation=0.
 - Or, 0 can be sent from Alice to Circuit.
- Expected score: 4 points.

Subtask 2

- $n = 1, 0 \leq m \leq 1$
- If $m = 1$, the answer is the number. Else, the answer should be 0.
- Alice sends the number (16 bits).
- Bob sends m (1 bit).
- Circuit performs 16 AND operations, and then output.
- Expected score: 8 points.

Subtask 3

- $n = 1, 0 \leq m \leq 1000$
- Answer is the number multiplies m .
 - It is unnecessary to implement multiplication in circuit.
Conducting addition for multiple times is enough for this subtask.
- Alice sends the number (16 bits).
- Bob sends m bits (contents does not matter, only to let circuit() directly know m)
- Circuit performs $m - 1$ times of addition
 - Assuming each addition needs 80 gates, there are 80000 gates in total.
- Expected score: 12 points.

How to implement addition?

- 1-bit adder
 - Input 3 bits: a, b, c
 - Output 2 bits: s1, s0 (the binary form of $A+B+C$ is S1 S0)
 - $S0 = a \text{ XOR } b \text{ XOR } c$
 - $S1 = (a \text{ AND } b) \text{ or } ((a \text{ XOR } b) \text{ AND } c)$
 - 5 gates
- 16-bit adder
 - Cascading 16 1-bit adders
 - For the three inputs of the 1-bit adders, two of them come from input numbers, and the other is the carry bit from the lower position.
 - The carry bit is initially 0, and the overflowed carry bit is discarded (equivalent to modulo 2^{16})
 - At most 80 gates (with optimization chances)

Subtask 4

- $n = 26$, names appearing in order, no duplicate letters
 - Means that Bob's input can be arranged as an $n \times n$ 0/1-matrix
- Alice sends the 26 numbers in the order from a to z (26×16 bits)
- Bob sends the 0/1-matrix (26×26 bits)
- Circuit performs AND and addition according to the matrix
 - Needs $26 \times 26 \times 16$ AND-s and 26×26 additions, which is far below the limit of 10^7 gates.
- Expected score: 24 points.
 - Can obtain 36 points combined with Subtasks 1~3.

Subtask 5

- Based on Subtask 4, but the names may not be ordered.
- Nevertheless, Bob knows all names.
 - His input can still be arranged as an $n \times n$ 0-1 matrix.
- If Alice and Bob follow the same order (e.g., lexicographical), numbers sent from Alice can still correspond to Bob's matrix.
 - However, this order may be different from the order of answer output.
- Alice sends an extra 26×26 0/1-matrix (where there are exactly one element 1 from each row and from each column), so that Circuit can rearrange the order of the answers.
- Expected score: 48 points.
 - Can obtain 60 points combined with Subtasks 1~3.

Subtask 6

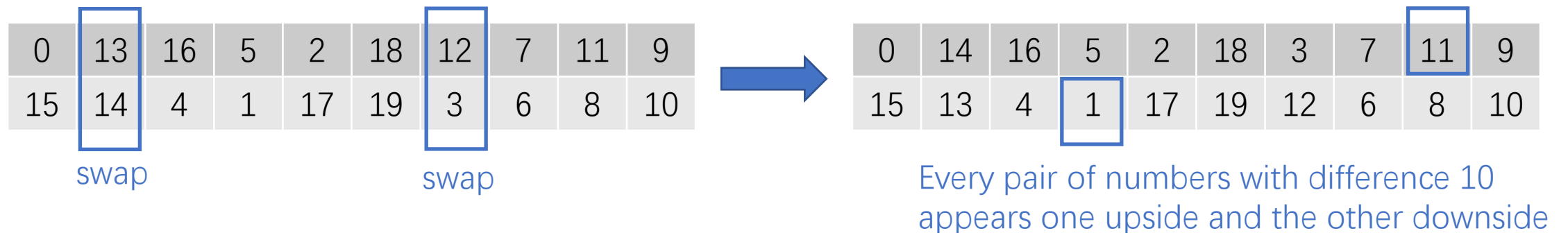
- n is not restricted to equal 26, and may be any value no more than 30.
- There is nothing special. You should pass this subtask unless your solution fails strangely.
 - The original purpose to place a subtask of $n = 26$ is to facilitate debugging.
- Expected score: 54 points.
 - Can obtain 66 points combined with Subtasks 1~3, which is the maximal score achieved during the competition.
- There also exists another approach of $16 \times 19 \times n \times m$, where the names are sent from Alice and Bob to Circuit. This can also get 66 points.

Subtasks 7~9

- Subtask 7: $n = 676, m \leq 1000$, names are in order.
- Subtask 8: names may be out of order.
- Assuming we can solve 7, then how to do 8?
 - We need a method to adjust the output order.
 - It is too inefficient to adopt an $n \times n$ 0/1-matrix!
- Considering there are $n!$ possible permutations for n elements, the lower bound is to use $\log_2 n! = O(n \log n)$ bits to encode a permutation, to be performed by Circuit
- We now provide a construction

How to implement an $O(n \log n)$ permutation?

- For example, we need to rearrange 20 elements to the ordered state.
- Split the array into the upper and the lower halves, each with 10 elements.
- Swap the elements up and down, so that every pair of numbers with difference 10 appears one upside and the other downside.
 - It can be proved that it is always possible to do so.



How to implement an $O(n \log n)$ permutation?

- Recursively process the upper half and the lower half, so that the ones-place becomes all correct.

0	14	16	5	2	18	3	7	11	9
15	13	4	1	17	19	12	6	8	10



0	11	2	3	14	5	16	7	18	9
10	1	12	13	4	15	6	17	8	19

Every pair of numbers with difference 10
appears one upside and the other downside

- Finally, swap the elements up and down, so that all elements become correct.

0	11	2	3	14	5	16	7	18	9
10	1	12	13	4	15	6	17	8	19



0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19

swap swap swap swap

How to implement an $O(n \log n)$ permutation?

- There are $n/2$ chances of optional swapping before recursion and after recursion, respectively. We can use $n/2$ bits each, to record whether to swap.
- It is similar when n is odd. There are $\lfloor \frac{n}{2} \rfloor$ chances of optional swapping before and after recursion, respectively. Not going into details here.
- Let $T(n)$ denote the number of bits to encode a permutation of n elements. Then $T(n) = 2 \lfloor \frac{n}{2} \rfloor + T\left(\lfloor \frac{n}{2} \rfloor\right) + T\left(\lceil \frac{n}{2} \rceil\right)$. According to the master theorem, $T(n) = O(n \log n)$.

Full-Score Algorithm

- Convert each person and each letter to a 4-tuple (U, V, W, T).
 - A person with name X and number Y converts to (X, X, Y, 0).
 - A letter with sender P and receiver Q converts to (P, Q, 0, 1).
 - (Assuming each string is converted to a 19-bit integer.)
- Sort all tuples by U first then V.
 - The sorting algorithm will be introduced later.
- Consider all tuples in order, maintaining a temporary variable C:
 - When meeting an element of T=0 (person), assign: $C \leftarrow (W \text{ of the current element})$
 - When meeting an element of T=1 (letter), assign: $(W \text{ of the current element}) \leftarrow C$
- At this point, all letters are assigned with the sender's number.

Full-Score Algorithm

- Next, sort all tuples by V first then T (descending).
- Consider all tuples in order, maintaining a temporary variable S (initially 0):
 - At an element of $T=1$ (letter), assign: $S \leftarrow S + (W \text{ of the current element})$
 - At an element of $T=0$ (person), assign: $(W \text{ of the current element}) \leftarrow S; S \leftarrow 0$
- At this point, all persons are assigned with the correct computation result.
- Sort all tuples by T first then U .
 - Now the first n elements are persons.
- Finally, adopt the $O(n \log n)$ permutation to rearrange the persons into the order of Alice's input, and output the result.

How to implement sorting?

- Conventional $O(n \log n)$ sorting (e.g., quicksort, mergesort, heapsort) cannot be directly implemented in circuits.
- But we can find that:
- Alice and Bob can sort their own elements in advance, respectively.
- Circuit only needs to merge two sorted arrays.
 - We can use $O(n \log n)$ in-place merge.
 - Note that the conventional $O(n)$ merging is still unimplementable in circuits.

Final Algorithm

- Alice sends to Circuit:
 - n 4-tuples of persons, sorted by name U .
 - The bits encoding the permutation which can rearrange n persons from ordered by U into ordered by Alice's input (i.e., output order).
- Bob sends to Circuit:
 - m 4-tuples of letters, sorted by sender U .
 - The bits encoding the permutation which can rearrange m letters from ordered by U into ordered by V .

Final Algorithm

- Circuit's computation:
 - After receiving the 4-tuples, merge two sorted arrays (first U then V).
 - Perform the first sequential scan, so that the letters are assigned with senders' numbers.
 - Undo the previous merge (which can be implemented by recording whether each comparison leads to a swap).
 - Apply Bob's permutation, to rearrange the letters from ordered by U into ordered by V.
 - Merge two sorted arrays again (first U then T descending).
 - Perform the second sequential scan, so that each person gets the correct result.
 - Undo the previous merge.
 - Apply Alice's permutation, to rearrange the persons into Alice's input order.
 - Output the answer.
- Expected score: 100 points.

Possible but Unnecessary Optimizations

- Constant propagation: e.g., $0 \text{ AND } x = 0$, $0 \text{ OR } x = x$
- “Undefined” propagation: e.g., $\text{undefined XOR } x = \text{undefined}$
- Dead circuit optimization: If the result of a computation gate is never used (directly or indirectly) by any output, the gate can be eliminated.
 - Enable less thinking when writing code.
- Duplicate circuit optimization: Merge two computation gates with the same operation type and dependency gates.
 - Help check for problems in the code.
- Reference answer needs about 10^7 gates, or about 8×10^6 after optimization.

The End